

1 INTRODUCCIÓN

La integración de aplicaciones al SISSEG requiere mecanismos estandarizados que permitan garantizar la seguridad, trazabilidad y control de acceso durante el intercambio de información.

El SISSEG V3.0 se compone de diversos componentes, tanto para conexiones Legacy como para integraciones actuales. Cuenta con un módulo de autenticación Keycloak, un módulo de autorización SISSEG-backend y un componente de acceso a los sistemas denominado SISSEG-frontend. En conjunto, forman el SISSEG V3.0.

1.1 OBJETIVO

Establecer un estándar de implementación para la integración con el SISSEGV3, esta integración contempla con el componente de autorización del SISSEG (Keycloak) como proveedor centralizado de autenticación y autorización en aplicaciones compuestas por:

- Frontend Angular.
- Backend Spring Boot.
- APIs REST protegidas mediante JWT.
- OpenID Connect (OIDC).
- OAuth 2.0.

El presente documento está orientado a equipos de desarrollo con diferentes niveles de experiencia, proporcionando lineamientos conceptuales, arquitectura de referencia, ejemplos de implementación y buenas prácticas.

1.2 ALCANCE

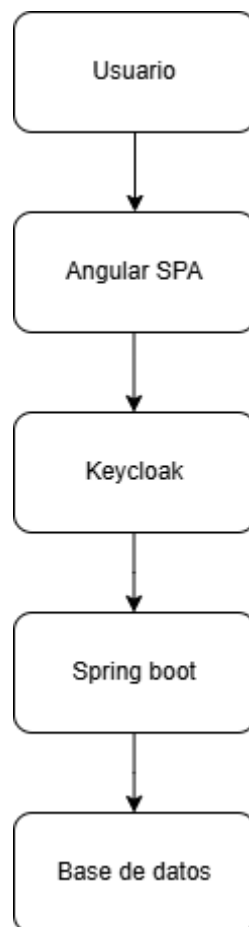
Este estándar aplica a aplicaciones que:

- Utilizan el componente de autorización del SISSEG (keycloak) como proveedor único de identidad.
- Requieren autenticación centralizada.
- Consumen APIs protegidas mediante JWT.
- Necesitan autorización basada en roles.
- Operan bajo arquitecturas SPA + API REST.

- Para el ejemplo se utilizo Angular 21 y Spring boot 4.0.6

2 ARQUITECTURA GENERAL

2.1 COMPONENTES



2.2 RESPONSABILIDADES

Componente	Responsabilidad
Autorización SISSEG - Keycloak	Autenticación y emisión de tokens
Angular	Gestión de sesión y consumo de APIs



MANUAL TÉCNICO PARA IMPLEMENTACIÓN DE COMPONENTE DE AUTORIZACIÓN DEL SISSEG (KEYCLOAK)

Código :
Revisión :
Fecha : 02/06/2026

Spring Boot Validación JWT y autorización

Base de Datos Persistencia de negocio

3 CONCEPTOS FUNDAMENTALES

Realm:

Contenedor lógico donde se administran:

- Usuarios.
- Roles.
- Clientes.
- Políticas de autenticación.

Ejemplo:

- OSIAPP
- OSIMOVIL

CLIENT:

Representa una aplicación registrada en Keycloak.

Ejemplos:

- sisseg
- nsrh-portal

ACCESS TOKEN:

Token JWT utilizado para acceder a recursos protegidos.

REFRESH TOKEN:

Permite renovar el Access Token sin volver a autenticarse.

ROLES:

Permiten controlar el acceso a funcionalidades. Los roles deben tener las siglas de la aplicación o el módulo antes del nombre del rol para poder diferenciar {SIGLA}-{NOMBRE ROL}.

Ejemplo:

- SST-ADMIN
- SFH-ESPECIALISTA
- NSGD-JEFE
- RTD-TRABAJADOR

4 REQUISITOS PREVIOS

- Inscribir la aplicación en el SISSEG, incluyendo la aplicación, sus módulos, páginas, roles y permisos correspondientes. Este proceso se realiza mediante la generación de un archivo Excel, el cual se remite al operador para su registro.
- Generar un cliente (Client) en Keycloak para la aplicación, configurando las URLs necesarias para la autenticación, autorización y redirección de usuarios.

5 CONFIGURACIÓN GENERAL EN COMPONENTE DE AUTORIZACIÓN DEL SISSEG (KEYCLOAK)

5.1 CLIENTE ANGULAR

Configuración recomendada:

- Client Type: Public
- Standard Flow: Enabled
- Authorization: Disabled
- PKCE: Enabled

5.2 CLIENTE BACKEND

Configuración recomendada:

- Client Type: Confidential
 - Service Accounts: Opcional
-

6 IMPLEMENTACIÓN FRONTEND ANGULAR

6.1 CONFIGURACIÓN POR AMBIENTE

```
export const environment = {  
  production: false,  
  
  keycloak: {  
    url: 'https://srvdesakclk.osinergrmin.gob.pe',  
    realm: 'OSIAPP',  
    clientId: 'sisseg',  
    redirectUri: 'http://localhost:4200'  
  },  
  
  backend: {  
    url: 'http://localhost:8080/kclk-backend'  
  }  
};
```

6.2 INICIALIZACIÓN DE KEYCLOAK

Ubicación:

main.ts

Ejemplo:

```
bootstrapApplication(App, {  
  providers: [  
    ...appConfig.providers,  
  
    provideKeycloak({  
      config: {  
        url: environment.keycloak.url,  
        realm: environment.keycloak.realm,  
        clientId: environment.keycloak.clientId  
      },  
      initOptions: {  
        onLoad: 'login-required',  
        pkceMethod: 'S256'  
      }  
    })  
  ]  
});
```

Consideraciones

- login-required obliga al usuario a autenticarse antes de acceder a la aplicación.
- S256 implementa PKCE para mayor seguridad.

6.3 AUTHSERVICE

Responsabilidades:

- Login.
- Logout.
- Obtener token.
- Obtener usuario.
- Obtener roles.
- Verificar autenticación.

Ubicación:

core/services/auth.ts

Ejemplo:

```
@Injectable({
  providedIn: 'root',
})
export class Auth {
  constructor(private keycloak: Keycloak) {

  }

  async login() {
    return this.keycloak.login();
  }

  async logout() {
    return this.keycloak.logout({
      redirectUri: window.location.origin
    });
  }
}
```

```
async isAuthenticated(): Promise<boolean> {  
    return !!this.keycloak.authenticated;  
}  
  
async getToken(): Promise<string> {  
    await this.keycloak.updateToken(30);  
    return this.keycloak.token ?? '';  
}  
}
```

Recomendación

Todos los componentes deben consumir AuthService.

No se recomienda utilizar KeycloakService directamente fuera de esta capa.

6.4 AUTH GUARD

Ubicación:

core/guards/auth.guard.ts

Responsabilidades:

- Verificar autenticación.
- Invocar login cuando sea necesario.
- Proteger rutas privadas.

Ejemplo:

```
export const authGuard: CanActivateFn = async () => {  
    console.log('guard ejecutado');  
    const authService = inject(Auth);  
  
    if (await authService.isAuthenticated()) {  
        return true;  
    }  
}
```

```
await authService.login();  
  
return false;  
};
```

Uso:

```
export const routes: Routes = [  
  {  
    path: '',  
    component: Inicio,  
    canActivate: [authGuard]  
  }  
];
```

Las rutas que necesitan permisos tendrán que utilizar CanActivate.

6.5 BEARER INTERCEPTOR

Ubicación:

core/interceptors/bearer.interceptor.ts

Responsabilidades:

- Adjuntar token JWT.
- Renovar token cuando sea necesario.
- Gestionar errores 401 y 403.

Ejemplo:

```
export const bearerInterceptor: HttpInterceptorFn = (req, next) => {  
  const auth = inject(Auth);  
  
  console.log('INTERCEPTOR EJECUTADO');  
  
  return from(auth.getToken()).pipe(  
    switchMap(token => {  
      return next(req.clone({  
        setHeaders: {
```



```
        Authorization: `Bearer ${token}`  
    }  
    }));  
  })  
);  
};
```

Registro en app.config.ts :

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes),  
  
    provideHttpClient(  
      withInterceptors([bearerInterceptor])  
    )  
  ]  
};
```

5.7 Gestión del Usuario

Se recomienda centralizar la lectura de información del token.

Ubicación:

core/utils/keycloak-user.util.ts

Ejemplo:

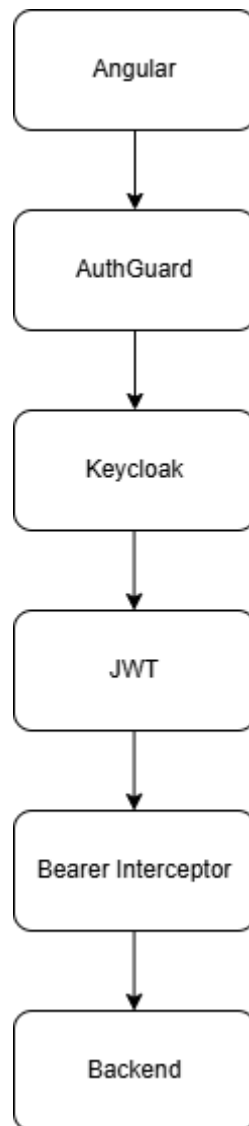
```
export function getUsername(  
  keycloak: Keycloak  
): string {  
  
  return keycloak.tokenParsed?.['preferred_username'] ?? '';  
}
```

Claims recomendados:

- preferred_username

- name
- email
- realm_access.roles

6.6 FLUJO FRONTEND



7 IMPLEMENTACIÓN BACKEND SPRING BOOT

7.1 DEPENDENCIAS

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-
server</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>

<dependency>
  <groupId>com.nimbusds</groupId>
  <artifactId>nimbus-jose-jwt</artifactId>
  <version>9.31</version>
</dependency>
```

7.2 CONFIGURACIÓN POR AMBIENTE

```
spring:
  application:
    name: kclk-backend

server:
  servlet:
    context-path: /kclk-backend

security:
  oauth2:
    resourceserver:
      jwt:
        issuer-uri:
${OAUTH2_PROVIDER:https://srvdesakclk.osinerghmin.gob.pe/realms/OSIAPP}

sisseg:
  url:
    profile: https://srvdesawssisseg.osinerghmin.gob.pe/sisseg-
backend/api/profile
```

7.3 SECURITYCONFIG

Ubicación recomendada:

shared/config/SecurityConfig.java

Responsabilidades:

- Configurar Resource Server.
- Definir rutas públicas y privadas.
- Configurar validación JWT.

Ejemplo:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final String issuerUriKeycloak;

    private final JwtGrantedAuthoritiesConverter defaultConverter = new
    JwtGrantedAuthoritiesConverter();

    public
    SecurityConfig(@Value("${security.oauth2.resourceserver.jwt.issuer-uri}")
    String issuerKeycloak) {
        this.issuerUriKeycloak = issuerKeycloak;
    }

    @Bean
    SecurityFilterChain securityFilterChain(
        HttpSecurity http
    ) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                    "/swagger-ui/**",
                    "/v3/api-docs/**"
                ).permitAll()
                .requestMatchers("/api/**")
```

```
        .authenticated()
    )
    .oauth2ResourceServer(
        oauth2 -> oauth2.jwt(
            jwt ->
jwt.decoder(jwtDecoder()).jwtAuthenticationConverter(jwtAuthenticationCon
verter()))
    );

    return http.build();
}

@Bean
public JwtDecoder jwtDecoder() {

    NimbusJwtDecoder keycloakDecoder = NimbusJwtDecoder
        //withJwkSetUri(kclkUrlBase +
"/realms/OSIAPP/protocol/openid-connect/certs")
        .withJwkSetUri( this.issuerUriKeycloak +
"/protocol/openid-connect/certs")
        .build();

    keycloakDecoder.setJwtValidator(
        //JwtValidators.createDefaultWithIssuer(kclkUrlBase +
"/realms/OSIAPP")
        JwtValidators.createDefaultWithIssuer(this.issuerUriKeycl
oack)
    );

    return keycloakDecoder;
}

@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {

    JwtGrantedAuthoritiesConverter grantedAuthoritiesConverter = new
JwtGrantedAuthoritiesConverter();
    grantedAuthoritiesConverter.setAuthorityPrefix("ROLE_");
    grantedAuthoritiesConverter.setAuthoritiesClaimName("roles");
}
```

```
        JwtAuthenticationConverter jwtAuthenticationConverter = new
JwtAuthenticationConverter();
        jwtAuthenticationConverter.setJwtGrantedAuthoritiesConverter(gran
tedAuthoritiesConverter);

        jwtAuthenticationConverter.setJwtGrantedAuthoritiesConverter(jwt
-> {

            Collection<GrantedAuthority> defaultAuthorities =
defaultConverter.convert(jwt);

            Map<String, Object> realmAccess =
jwt.getClaim("realm_access");
            if (realmAccess == null || !realmAccess.containsKey("roles"))
{
                return defaultAuthorities;
            }

            @SuppressWarnings("unchecked")
            List<String> roles = (List<String>) realmAccess.get("roles");

            Collection<GrantedAuthority> authorities = roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" +
role.toUpperCase())).collect(Collectors.toList());

            if (defaultAuthorities != null) {
                authorities.addAll(defaultAuthorities);
            }

            return authorities;
        });

        return jwtAuthenticationConverter;
    }
}
```

En una clase Util podemos descomponer el token para obtener información del usuario como el nombre.

Ubicación:

shared/Utils/UserUtils.java

Ejemplo:

```
public static Optional<String> getCurrentUsername() {
    Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();
    if (authentication == null || !authentication.isAuthenticated())
{
        return Optional.empty();
    }
    if (authentication.getPrincipal() instanceof Jwt jwt) {
        String preferredUsername =
jwt.getClaimAsString("preferred_username");
        if (preferredUsername != null) {
            return Optional.of(preferredUsername);
        }
    }
    return Optional.ofNullable(authentication.getName());
}
```

Uso:

```
String username = UserUtils.getCurrentUsername().orElse(null);
```

7.5 AUTENTICACIÓN Y AUTORIZACIÓN

Autenticación

Valida:

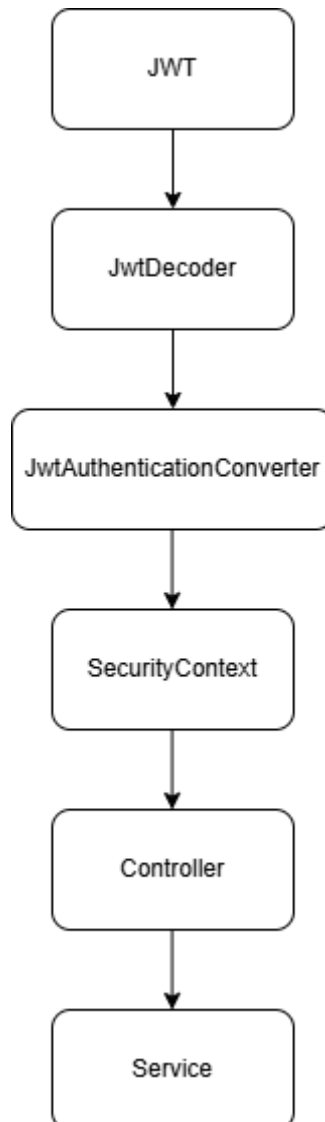
- Firma.
- Expiración.
- Issuer.
- Estructura JWT.

Autorización

Valida:

- Roles.
- Permisos funcionales.
- Reglas de negocio.

7.6 FLUJO BACKEND



8 INTEGRACIÓN CON SISSEG

8.1 OBTENER ROLES

Mediante la creación de una clase de servicio, es posible obtener los roles asociados al usuario autenticado en la aplicación. Para ello, únicamente se deben especificar los parámetros **siglaapp** y **mdloapp**, correspondientes a las siglas de la aplicación y al módulo al que se está accediendo, respectivamente. El parámetro **mdloapp** es opcional.

```
@Override
public SissegResponseWrapperModel obtenerPerfilSisseg(String
username, String siglaAplicacion, String moduloAplicacion, String token)
{
    Map<String, String> requestBody = new HashMap<>();
    requestBody.put("username", username);
    requestBody.put("siglaapp", siglaAplicacion);
    requestBody.put("mdloapp", moduloAplicacion);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.setBearerAuth(token);

    HttpEntity<Map<String, String>> requestEntity = new
HttpEntity<>(requestBody, headers);
    ResponseEntity<SissegResponseWrapperModel> response =
restTemplate.exchange(
        securityProperties.getUrl().getProfile(),
        HttpMethod.POST,
        requestEntity,
        SissegResponseWrapperModel.class
    );

    return response.getBody();
}
```

Posteriormente, mediante otro componente, se pueden obtener los roles necesarios para realizar las validaciones correspondientes en los endpoints.

```
@Component("sissegAuth")

public class SissegAuthorizationService {

    @PostConstruct
    public void init() {
        System.out.println("SissegAuthorizationService cargado");
    }

    private final SissegService sissegService;

    public SissegAuthorizationService(SissegService sissegService) {
        this.sissegService = sissegService;
    }

    public boolean hasAnyRole(String... requiredRoles) {
        System.out.println(requiredRoles.toString());
        Set<String> required = Arrays.stream(requiredRoles)
            .filter(role -> role != null && !role.isBlank())
            .map(role -> role.toUpperCase(Locale.ROOT))
            .collect(Collectors.toSet());

        if (required.isEmpty()) {
            return true;
        }

        Set<String> userRoles = resolveCurrentSissegRoles();
        boolean allowed =
userRoles.stream().anyMatch(required::contains);
        return allowed;
    }

    private Set<String> resolveCurrentSissegRoles() {
        String username = UserUtils.getCurrentUsername().orElse(null);
        String token = UserUtils.getToken();

        if (username == null || username.isBlank() || token == null ||
token.isBlank()) {
            return Collections.emptySet();
        }
    }
}
```

```
    }

    try {
        SissegResponseWrapperModel response =
sissegService.obtenerPerfilSisseg(username, "PR-SISSEG", "MODULO PRUEBA"
, token);
        if (response == null || response.roles() == null) {
            return Collections.emptySet();
        }

        return response.roles().stream()
            .map(SissegRolModel::nombre)
            .filter(role -> role != null && !role.isBlank())
            .map(role -> role.toUpperCase(Locale.ROOT))
            .collect(Collectors.toSet());
    } catch (Exception ex) {
        return Collections.emptySet();
    }
}
```

En los endpoints, únicamente es necesario agregar la etiqueta correspondiente para filtrar el acceso según el rol que se desee aplicar.

```
@GetMapping("/getRoles")
@PreAuthorize("@sissegAuth.hasAnyRole('PRUEBA-SISSEG')")
public ResponseEntity<SissegResponseWrapperModel> getRoles() {
    String username = UserUtils.getCurrentUsername().orElse(null);
    String token = UserUtils.getToken();

    return
ResponseEntity.ok(sissegService.obtenerPerfilSisseg(username, "PR-
SISSEG", "MODULO PRUEBA" , token));
}
```

9 MANEJO DE ERRORES

Frontend

Error 401

Acciones recomendadas:

- Logout
- Limpiar sesión
- Redireccionar login

Error 403

Mostrar acceso denegado

o

Cerrar sesión

Backend

401 Unauthorized

Causas:

- Token inválido.
- Token expirado.
- Issuer incorrecto.

403 Forbidden

Causas:

- Falta de roles.
- Falta de permisos.

10 BUENAS PRÁCTICAS

Seguridad

- Utilizar PKCE.
- Utilizar HTTPS.
- No almacenar tokens en localStorage.
- Externalizar secretos.
- Configurar correctamente Redirect URIs.

Desarrollo

- Centralizar autenticación en AuthService.
- Centralizar lectura de usuario.
- Centralizar lectura de roles.
- Evitar lógica de seguridad duplicada.

Operación

- Un solo realm por aplicación.
- Un único issuer-uri.
- Rotación periódica de secretos.
- Validación continua de roles.

11 ANEXOS

11.1 PARAMETROS DE COMPONENTE DE AUTORIZACIÓN DEL SISSEG (KEYCLOAK)

- Ejemplo para el ambiente DESA:
keycloak-url: <https://srvdesakclk.osinergrmin.gob.pe/>
- Real: OSIAPP

11.2 PARAMETROS PARA SISSEG

- Sisseg-url: <https://srvdesawssisseg.osinergrmin.gob.pe/sisseg-backend>
- Endpoint para roles: api/profile
- Parametros:
 - Username: Username del usuario logueado
 - Siglaapp: Siglas de la aplicación
 - Mdloapp: Modulo que se está
 - consumiendo (opcional)